Ders 9 Genişletilmiş Notu: PPM ve Sözlük Tabanlı Sıkıştırma: LZ77, LZ78 ve LZW

Ders 9 Genişletilmiş Notu: PPM ve Sözlük Tabanlı Sıkıştırma: LZ77, LZ78 ve LZW

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. PPM'in Tekrarı: Order-N Modeli ve İçerik Tabanlı Olasılık
- 2. Statik ve Adaptif İçerik Tabanlı Modeller
- 3. Lipogram
- 4. Order-N Adaptive Context-Based Model ve N'in Seçimi
- 5. Order-4 Context Tablosunun Kurulması
- 6. Switch İşlemi ve Escape Sembolü
- 7. Order-2 Tablo ve Escape Frekansları
- 8. Escape ile Kodlama Senaryoları
- 9. PPM'in Versiyonları: PPMA, PPMB, PPMC
- 10. Context'lerin Trie ile Tutulması
- 11. Sözlük Tabanlı Yöntemlere Giriş
- 12. Statik Sözlük Örneği ve Sıkıştırma Koşulu
- 13. String Sıkıştırma: Birden Fazla Sembolü Birlikte Kodlamak
- 14. LZ ve Türev Algoritmalar
- 15. LZ77: Kayan Pencere
- 16. LZ77 Örnekleri ve Kod Çözme
- 17. LZ77'de Alan Uzunlukları ve Geliştirmeler
- 18. LZ78
- 19. LZW
- Örnek Sorular

1. PPM'in Tekrarı: Order-N Modeli ve İçerik Tabanlı Olasılık

Hoca derse, bir önceki derste başlanan PPM algoritmasını özetleyerek başladı. PPM, istatistiksel yöntemlerin son algoritmasıdır; bu ders onu bitirip sözlük tabanlı yöntemlere geçecektir. Hoca, derste öncelik verdiği yöntemlerin günlük hayatta kullanılan sıkıştırma programlarının ve resim formatlarının içinde de yer alan yöntemler olduğunu, bunların ileride tekrar karşılarına çıkacağını söyledi.

Slayttaki bilgiler (ekrandan, 00:10:00):

- PPM, J. Cleary ve I. Witten tarafından ortaya atılmış, A. Moffat tarafından daha iyi hale getirilmiştir.
- PPM (slayttaki adıyla Prediction with Partial String Matching), **order-N Markov** modelini kullanan içerik (context) tabanlı bir sıkıştırma yöntemidir.
- Model, sembole bir olasılık değeri atamadan önce sembolün içeriğini (context'ini) inceler.
- Kodlayıcı ve kod çözücü metnin ilerisine erişemediğinden, her ikisi de geçmişteki metnin içeriği ile ilgilenir.
- Pratikte bir sembolün içeriği, kendisinden önce gelen N adet semboldür.

> **Ders sonrası not:** Yöntemin literatürdeki adı büyük olasılıkla “Prediction by Partial Matching”tir; slayttaki “Prediction with Partial String Matching” aynı yöntemi anlatır.

İstatistiksel yöntemlerde her zaman sembollerin olasılık değerlerine ihtiyaç vardır. PPM de bir istatistiksel yöntemdir, ama olasılıkları farklı çıkarır. Klasik yöntemde bir A sembolünün olasılığı, metinde kaç kez geçtiğinin metindeki toplam karakter sayısına oranıdır. Örneğin metinde 1217 sembol varsa ve bunların 34’ü q ise q’nun olasılığı $34/1217$’dir. Order-N modelinde ise sembolün geçmişi kullanılır. Hoca tahtada bir metin üzerinde gösterdi: bir A’nın olasılığı hesaplanırken “geçmişinde T olan A’ların” olasılığı, başka bir A için “geçmişinde R olan A’ların” olasılığı kullanılır. Hocaya göre istatistiksel yöntemlerden tek farkı budur: geçmişe bakar.

İçerik tabanlı istatistiksel modellerde sembolün olasılığı context’e bakarak verilir. Örneğin h’nin olasılığı ayrı, t’nin olasılığı ayrıdır; ama t ve h’nin birlikte kullanılma olasılığı çok daha farklı olabilir, çünkü h geçmişinde bir sürü farklı karakterle birlikte kullanılmıştır.

2. Statik ve Adaptif İçerik Tabanlı Modeller

Yöntemler statik ve dinamik diye ikiye ayrılır.

- **Statik:** önceden verilmiş bir korpus içinde, N değeri ne ise (2, 3, …) bütün context’ler önceden çıkarılır. Olasılıklarına göre kodlar atanır ve doküman bu sabit kodlarla sıkıştırılır.
- **Adaptif (dinamik):** önceden hazırlanmış bir kod tablosu yoktur. Context’ler ve olasılıklar algoritma çalışırken çıkarılır. Örneğin bir A’nın olasılığı verilirken o noktaya kadar kaç kez “ra” geldiyse olasılık oradan çıkarılır. Bir dokümanda herhangi bir ikilinin olasılık değeri başta neyse sonda aynı değildir; dokümanda ilerledikçe değişir.

Olasılıklar dinamik olarak değiştiğine göre sembollere verilen kodlar da dinamik olarak değişir. Hoca bunu dinamik Huffman’a benzetti: orada da kodlar doküman işlenirken sürekli değişebiliyordu. İçerik tabanlı yöntemde fark, olasılıkların tek tek karakterler için değil, context’e göre (geçmişi 2 ise 2, 3 ise 3 sembole bakarak) hesaplanmasıdır. Bu yüzden hem olasılıkların hem kodların defalarca hesaplanması gerekir.

3. Lipogram

Hoca bir önceki derste “lipogram” konusunda kaldıklarını söyledi ve bunu genel bilgi olarak anlattı (ekrandan, 00:17:45).

Slayttaki tanım: bir metinde kullanılan karakterlerin olasılık değerleri birbirinden çok ayrıksa bu durum «lipogram» olarak adlandırılır. Normal bir metinde olasılıklar birbirine yakındır; örneğin A’nın olasılığı 0,8 ise E’ninki 0,79 gibidir, değişim vardır ama varyans küçüktür. Lipogramda ise A’nın olasılığı 0,8 iken sonraki sembollerin olasılığı 0,01, F’ninki 0,001 gibi olabilir; varyans büyür.

Slayttaki örnekler:

- En iyi bilinen lipogram Ernest Vincent Wright’ın **Gadsby** romanıdır. İçerisinde E harfi yoktur. E’nin olasılığı sıfır olur, diğer harflerin olasılık dağılımı dengesizleşir.
- Walter Abish’in **Alphabetical Africa** romanı 52 bölümden oluşur. 1. bölümde bütün kelimeler a ile, 2. bölümde a ve b ile başlar; 26. bölümde bütün kelimeler a-z ile başlar. Geriye kalan 26 bölümde aynı şekilde azalarak devam eder.

Böyle bir dokümanda karakterlerin olasılıkları birbirinden ayrık olur. Hoca, lipogram olan bir dokümanın iyi sıkıştırılmayacağını, iyi sıkıştırma için olasılık değerlerinin birbirine yakın olması gerektiğini söyledi. Doğal dilde yazılmış normal bir dokümanda böyle bir durum çıkmaz; bu eserlerde yazar özellikle bazı karakterleri kullanmamak için kendini zorlar.

> **Ders sonrası not:** Hoca Gadsby’de “büyük E harfi” geçmediğini söyledi. Bu romanda büyük olasılıkla e harfi küçük ya da büyük hiç kullanılmamıştır. Lipogram kelimesi genelde bir ya da birkaç harfin bilerek hiç kullanılmadığı metin için kullanılır; slayttaki tanım bunun olasılık dağılımına etkisini anlatır.

4. Order-N Adaptive Context-Based Model ve N'in Seçimi

PPM yöntemine **Order-N Adaptive Context-Based Model** de denir: hem adaptif (dinamik) olarak değişir hem de context'e bağlıdır; context'in ne kadar geçmişine bakılacağını N söyler. Slayttaki bilgiler (ekrandan, 00:20:00):

- Input stream'den gelecek olan S sembolünü okur ve mevcut order-N modeline göre kendisinden önce gelmiş olan N sembolü bulur ve S'nin olasılığını tahmin eder.
- Teoride N değeri çok büyük seçilir (N = 20 bin), ancak pratikte bunun dezavantajı vardır.

Hoca örnekle açıkladı: sıra bir C sembolüne gelmiş ve N = 5 olsun. C'nin 5 geçmişine bakılır; daha önce böyle bir pattern gelmişse onun olasılık değeri kullanılır. Gelmemişse N biraz azaltılır (switch edilir), "böyle bir şey hiç karşıma çıkmış mı" diye tekrar bakılır. C hiç görülmemişse bu iniş en alta kadar sürer ve C'ye yine bir kod değeri verilir.

N = 20.000 alınırsa ilk 20.000 karaktere gelinceye kadar bütün karakterler ASCII olarak, yani sıkıştırılmadan kodlanır. Hoca bunun sıkıştırma yapan biri için anlamsız olduğunu söyledi. N büyüdükçe context sayısı da patlar:

- Semboller 7 bitlik olsun: $2^7 = 128$ farklı sembol vardır. Order modeli kullanılmazsa 128 sembolün olasılığı tutulur.
- Order 2'de 128 sembolün ikili kombinasyonları gelebilir: $128^2 = 16.384$ ikili.
- Order 3'te $128^3 = 2.097.152$ farklı kombinasyon gelebilir.
- N = 20.000 için 128^{20000} kombinasyon olur; hoca bununla iş yapılamayacağını söyledi. Ayrıca her metnin 20.000 karakterden uzun olması gerekir ve 20.000 karakterlik geçmişe sahip bir kombinasyon belki bir kez gelir, bir daha gelmez.

Hoca bir de tarih kitabı örneği verdi: ortaokul ve lise tarih kitaplarında her bölüm başka bir çağı anlatır. Farklı bölümlerde geçen kelimeler ve sembol dağılımları birbirinden çok farklı olduğundan N büyük alınırsa çakışmalar çok azalır ve PPM ile başarılı bir sonuç alınamaz.

Slayta göre pratikte N 2 ile 10 arasında alınır. Hoca, akademik çalışmalarda N'in 5'i geçtiğini görmediğini, büyük çoğunluğun 3'e kadar çıktığını söyledi. Daha büyük N'de kombinasyonların bir veri yapısında tutulması ve onlara erişilmesi algoritmanın süresini uzatır.

5. Order-4 Context Tablosunun Kurulması

Hoca, slayttaki hazır tabloyu (Table 2.72) üzerine kalemle yazarak adım adım yeniden kurdu (ekrandan, 00:30:00 ve 00:40:00). Soru iki şıklıdır: (a) verilen girdiyi order-4 modeline göre PPM ile kodlamak için context'leri ve frekanslarını çıkar; (b) tabloya yeni bir karakter gelince tabloyu güncelle.

Girdi: «xyzzxyxyzzx». Order 4 olduğu halde tabloda Order 4, 3, 2, 1 ve 0 olmak üzere beş kolon vardır, çünkü aranan order-4 context'i daha önce hiç gelmemişse Order 3'e, orada da yoksa 2, 1 ve 0'a switch edilir. Order 0'da da yoksa sembol ilk kez gelmiştir ve tabloya yeni sembol olarak eklenir.

Kurulum adımları (hocanın anlatımından):

1. **x:** tablo boş, x'in geçmişi yok. Order 0'a x 1 yazılır.
2. **y:** Order 0'da y yok, y 1 eklenir. y'nin yalnız bir geçmişi vardır (x); Order 1'e x→y 1 eklenir.
3. **z:** Order 0'a z 1; Order 1'e y→z 1; Order 2'ye xy→z 1.
4. **z:** Order 0'da z 2 olur; Order 1'e z→z 1; Order 2'ye yz→z 1; Order 3'e xyz→z 1.
5. **x:** Order 0'da x 2; z→x 1, zz→x 1, yzz→x 1, Order 4'e xyzz→x 1.
6. **y:** Order 0'da y 2; x→y 2; zx→y 1; zzx→y 1; yzzx→y 1.
7. **x:** Order 0'da x 3; y→x 1; xy→x 1; zxy→x 1; zzxy→x 1.
8. **y:** Order 0'da y 3; x→y 3; yx→y 1; xyx→y 1; zxyx→y 1.
9. **z:** Order 0'da z 3; y→z 2; xy→z 2; yxy→z 1; xyxy→z 1.
10. **z:** Order 0'da z 4; z→z 2; yz→z 2; xyz→z 2; yxyz→z 1.
11. **x:** Order 0'da x 4; z→x 2; zz→x 2; yzz→x 2; xyzz→x 2.

Sonuç tablo (a) (ekrandan, 00:30:00; python ile doğrulandı):

Order 4	Order 3	Order 2	Order 1	Order 0
xyzz→x 2	xyz→z 2	xy→z 2	x→y 3	x 4
yzzx→y 1	yzz→x 2	xy→x 1	y→z 2	y 3
zzxy→x 1	zzx→y 1	yz→z 2	y→x 1	z 4
zxyx→y 1	zxy→x 1	zz→x 2	z→z 2	
xyxy→z 1	xyx→y 1	zx→y 1	z→x 2	
yxyz→z 1	yxy→z 1	yx→y 1		

(b) şıkkında girdiye bir z daha gelir. Order 0'da z 5 olur. z'nin önünde x vardır: Order 1'de x→z yoktur, 1 ile eklenir. zx→z, zzx→z ve yzzx→z de tabloda yoktur; her biri 1 ile eklenir (ekrandan, 00:40:00).

Hoca, PPM'e başlamadan önce istenen order-N modeline göre dokümanın kullanım frekanslarının çıkarılması gerektiğini, frekanslardan olasılıkları hesaplamanın çok kolay olduğunu söyledi. Bu yüzden yöntem aslında iki fazlıdır: önce frekanslar ve olasılıklar çıkarılır; sonra bu olasılıklar aritmetik kodlamadaki gibi $[0, 1)$ aralığına dağıtılır ve aritmetik kodlama çalıştırılır. Fark, olasılıkların statik olarak değil, order-N modeliyle çıkarılmasıdır.

> **Hocanın vurgusu:** Hoca tabloyu kendiniz, onun kalemle yaptığı gibi baştan kurmanızı ve slayttakiyle karşılaştırmanızı istedi: "Ben nasıl çıkardıysam sizin de çıkarmış olmanız gerekir."

6. Switch İşlemi ve Escape Sembolü

PPM'in çalışma prensibi: sıfır olasılıklı bir context geldiğinde onu daha kısa uzunluktaki bir context ile yer değiştirir. Hoca bunu statik kullanım üzerinden anlattı. Tablolar bir kez oluşturulmuş, frekanslar çıkarılmıştır. Yeni bir doküman geldiğinde aynı tablo kullanılır. Örneğin «k y z z x» girişinde x için Order 4'te kyzz→x aranır ve bulunamaz; bu durumda Order 3'e switch edilir ve orada aranır. Bulunursa o pattern'in olasılık değeri kullanılır; bulunamazsa Order 2, Order 1'e kadar devam edilir.

Algoritmanın adımları:

- Order-N context ile başlanır. Gelen S sembolü, order-N modeline göre önceden gelmiş C context'i içinde aranır.
- Bulunamazsa, S'yi izleyen C context'i elde mevcut değildir; N bir azaltılır ve N-1'e switch edilir.
- PPM kodlayıcı bu switch işlemini Order 0 context'ine gelinceye kadar sürdürür.
- Order 0'da da yoksa (hocanın örneğiyle Fransızcadaki aksanlı bir e gibi hiç görülmemiş bir sembol), sembole sabit bir olasılık atanır: $1/$alfabenin boyu. Örneğin bugüne kadar 3 bağımsız karakter gelmişse ve bu dördüncüsüyse olasılık $1/4$ olur.
- Bulunan P olasılıklı her sembol, kodlanmak için aritmetik kodlayıcıya gönderilir.

Kod çözme de aritmetik kodlamadaki gibidir. Ama kodlayıcı nerede switch ettiğini bilir, kod çözücü bilmez. Bu bilgiyi karşı tarafa göndermek için **escape** sembolü kullanılır. Her order modelinin, her context'in escape sembolü ayrıdır. Kodlayıcı, switch işlemini gerçekleştirmeden önce escape sembolünün olasılığını aritmetik kodlayıcıya gönderir; yani escape de aritmetik kodlamada bir sembol gibi kodlanır.

7. Order-2 Tablo ve Escape Frekansları

İşlemler çok uzadığı için hoca escape örneğini order-2 modeliyle yaptı. Girdi: «assanissimassa» (ekrandan, 00:50:00). Hoca bu tür anlamsız kelimelerin bilerek seçildiğini açıkladı: "okul" gibi normal bir kelimede hiçbir harf tekrar etmez; algoritmanın gösterilebilmesi için tekrar eden harfler ve pattern'ler gerekir.

Tablo aynı mantıkla kurulur: a gelir (Order 0'da a 1); s gelir (s 1, a→s 1); s gelir (s 2, s→s 1, as→s 1); a gelir (a 2, s→a 1, ss→a 1) ve bu böyle devam eder. Hoca tahtada sayarken birkaç yeri yanlış yere yazdığını fark edip düzeltti; sonuç tablo slayttakiyle aynıdır.

Escape frekansları: her context ayrı bir blok gibi düşünülür (geçmişi s olanlar bir blok, geçmişi ss olanlar bir blok …). Bir bloktaki escape frekansı, o context'i izleyen farklı sembol sayısıdır.

Örneğin ss'yi a ve i izler, escape 2'dir; s'yi s, a ve i izler, escape 3'tür; Order 0'da 5 farklı sembol vardır, escape 5'tir. Olasılık, frekansın blok toplamına (escape dahil) bölünmesiyle bulunur.

Slayttaki tablo (Table 2.73, ekrandan, 00:50:00; frekanslar python ile doğrulandı):

Order 2	f	p	Order 1	f	p	Order 0	f	p
as→s	2	2/3	a→s	2	2/5	a	4	4/19
esc	1	1/3	a→n	1	1/5	s	6	6/19
ss→a	2	2/5	esc	2	2/5	n	1	1/19
ss→i	1	1/5	s→s	3	3/9	i	2	2/19
esc	2	2/5	s→a	2	2/9	m	1	1/19
sa→n	1	1/2	s→i	1	1/9	esc	5	5/19
esc	1	1/2	esc	3	3/9			
an→i	1	1/2	n→i	1	1/2			
esc	1	1/2	esc	1	1/2			
ni→s	1	1/2	i→s	1	1/4			
esc	1	1/2	i→m	1	1/4			
is→s	1	1/2	esc	2	2/4			
esc	1	1/2	m→a	1	1/2			
si→m	1	1/2	esc	1	1/2			
esc	1	1/2						
im→a	1	1/2						
esc	1	1/2						
ma→s	1	1/2						
esc	1	1/2						

Toplamlar: as bloğu $2+1=3$; ss bloğu $2+1+2=5$; s bloğu $3+2+1+3=9$; a bloğu $2+1+2=5$; Order 0 $4+6+1+2+1+5=19$. Bütün context'lerin olasılıkları elde olunca aritmetik kodlama çalıştırılabilir: bunlar sembollermiş gibi $[0,1)$ aralığına dağıtılır.

8. Escape ile Kodlama Senaryoları

Senaryo: metin «sa» ile bitiyor (son order-2 context'i sa) ve sırada bir sembol var (ekrandan, 01:00:00). Bit sayıları $-\log_2 P$ ile bulunur (python ile doğrulandı).

1. n gelirse. Order 2'de geçmişi sa olan bir n var mı? Var: sa→n, $P=1/2$. Aritmetik kodlayıcıya $1/2$ gönderilir. $-\log_2(1/2)=1$ bit.

2. s gelirse. Geçmişi sa olan blokta s yoktur. Switch edilmesi ve bunun kod çözücüye gösterilmesi gerekir: sa bloğunun escape'i $(1/2)$ gönderilir. Sonra Order 1'de geçmişi a olan bir s aranır: a→s var, $2/5$ gönderilir. Toplam: $-\log_2(1/2\cdot 2/5)\approx 2{,}32$ bit. Hoca bunun ortalama olduğunu, sembolün 2 ya da 3 bitle kodlanabileceğini söyledi.

3. m gelirse. sa bloğunda m yok: escape $1/2$. Order 1'de geçmişi a olan bir m yok: a bloğunun escape'i $2/5$. Order 0'da m var: $1/19$. Toplam: $-\log_2(1/2\cdot 2/5\cdot 1/19)\approx 6{,}57$ bit.

> **Ders sonrası not:** Hoca 01:02:41'de bu senaryoda $2/5$'i yanlış yazdığını, $1/2$ olması gerektiğini söyledi. Slayttaki 6,57 bit ve tablo, a context'inin escape'i olan $2/5$'i kullanır (python ile doğrulandı); $1/2$ ile sonuç 6,25 bit olurdu. Doğrusu $2/5$'tir.

4. d gelirse. sa bloğunda d yok: escape $1/2$. Şimdiye kadar d hiç kullanılmadığı için a bloğunda da yok: escape $2/5$. Order 0'da da yok: escape $5/19$. Yeni gelen sembol **Order** -1'e atanır ve olasılığı $1/$alfabe boyu'dur; örnekte alfabe 28 semboldür: $1/28$. Toplam: $-\log_2(1/2\cdot 2/5\cdot 5/19\cdot 1/28)\approx 9{,}05$ bit.

Hoca, tek bir d'yi kodlamak için 9,05 bit kullanıldığını, ASCII ile kodlansaydı 8 bit olacağını vurguladı. Demek ki dokümanın ilk başlarında, context'ler henüz oluşmadığında pek sıkıştırma olmaz. Context'ler arttıkça ve d "hayata girdikçe" sıkıştırma iyileşir. Model dinamikse d tabloya eklenir: Order 0'da d 1 olur, escape 6 olur ve ikili context'leri varsa onlar da eklenir. Tablolar algoritma çalıştıkça sürekli değişir.

> **Ders sonrası not:** Hoca 01:05:37'de d eklendikten sonra Order 0 olasılıklarının 20'ye bölüneceğini söyledi. Sembol frekansları $4+6+1+2+1+1=15$ ve escape 6 olduğundan payda büyük olasılıkla 21'dir.

9. PPM'in Versiyonları: PPMA, PPMB, PPMC

Slayttaki bilgiler (ekrandan, 01:08:50):

- PPM'in üç farklı versiyonu vardır. Bu modellerin yalnız escape sembollerinin kullanımında farklılıklar vardır; sıkıştırma oranlarında da çok küçük farklar oluşur.
- Bir önceki gördüğümüz yöntem **PPMC**'dir.
- **PPMA**'da toplam frekans n ise escape sembolüne $1/(n+1)$, diğer sembollere x/n olasılığı atanır.
- **PPMB**'de C context'inin içinde s iki kez göründükten sonra C context'ini izleyen S sembolüne bir olasılık atanır. Eğer «abc» context'i üç kez ve bunu izleyen x iki kez, y de bir kez görünmüşse x için $(2-1)/3$, y için $(1-1)/3$ olasılığı atanır.

Hoca PPMB'yi şöyle açıkladı: bir sembol bir context içinde yalnız bir kez geçmişse onun order 1, 2, 3 context'leri tabloya yazılmaz; sembol iki kez geçtikten sonra eklenir. Versiyonların farkları işlem adımlarındaki küçük parametrelerdir. Sonuçlar birbirine çok yakın çıkar; metnin içinde çok bariz farklı içerikler varsa bu nüanslar %1-2 gibi bir iyileşme sağlar. En çok kullanılan ve popüler olanı PPMC'dir.

10. Context'lerin Trie ile Tutulması

Context'ler ve frekanslar çıkarıldıktan sonra onlara hızlı erişmek gerekir. Bunun için algoritma derslerinden bilinen **TRIE** ağaç yapısı kullanılır: root'tan başlayıp yapraklara kadar izlenen her yol bir context'e karşılık gelir. Hoca, tabloyu sıralı dizilerde tutup her seferinde sıralı arama yapmanın verimsiz olduğunu ("Önce buradaki dizide sıralı ara, sonra burada sıralı ara ..."), tabloyu baştan ağaç üzerinde oluşturmanın erişimi kolaylaştırdığını söyledi.

Slayttaki örnek «zxzyzxxyzx» için on adımda kurulan trie'lerdir (Figure 2.74, ekrandan, 01:10:00). Hocanın anlatımıyla ilk adımlar:

1. **z:** root'tan inilir, z,1 düğümü eklenir.
2. **x:** root'a x,1 eklenir. x'in öncesinde z vardır; z düğümünün altına x,1 eklenir (zx).
3. **z:** root'taki z 2 olur. z'nin geçmişinde x vardır: x'in altına z,1 (xz). Ayrıca z→x yolunun altına z,1 eklenir (zxz).
4. **y:** root'a y,1 eklenir; z'nin altına y,1 (zy); x→z yolunun altına y,1 (xzy).
5. **z:** root'taki z 3 olur; y'nin altına z,1 (yz); z→y yolunun altına z,1 (zyz).

Her adımda frekansı artan ya da yeni eklenen düğümler slaytta yuvarlak içine alınmıştır. Onuncu adımdan sonra root'un altında x,4; y,2; z,4 düğümleri vardır. Tabloya yerleştirmek ile ağaca yerleştirmek aynı bilgidir; ağaç yalnız erişimi kolaylaştırır. Ağacın derinliği order ile sınırlıdır.

> **Ders sonrası not:** Hoca order 3 için ağacın seviyesinin 3 olacağını söyledi. Slayttaki trie'lerin derinliği 3'tür (2 sembollük context + sembol); bu trie'ler büyük olasılıkla order-2 içindir ve order-N için derinlik $N+1$ olur.

Algoritmayı basitleştirmek, switch işlemlerini kolay yapmak için ağaca **pointer**'lar eklenir. Örneğin order-2 modelinde «s a s» geldiğinde önce en büyük context'e bakılır: geçmişi «as» olan s düğümü. Orada bulunamazsa context küçültülüp yalnız «s»ye geçilmesi gerekir; düğümdeki pointer doğrudan bu kısa context'in düğümünü gösterir. Böylece her switch'te order bir azaltılarak tek sembole kadar inilebilir.

11. Sözlük Tabanlı Yöntemlere Giriş

Aradan sonra hoca "bambaşka bir dünyaya" geçti. Şimdiye kadar istatistik tabanlı yöntemler görülmüştü, şimdi sözlük tabanlı yöntemler görülecekti.

Hoca önce bir uyarı yaptı: sözlük denince bir dile ait sözlük ya da anlamlı kelimelerden oluşan bir liste akla gelmemelidir. Bu sözlük bir takım context'ler içerir ve bunlar belirli formattaki token'lar yardımıyla sözlükte toplanır.

İki yaklaşımın farkı:

- İstatistiksel sıkıştırma yöntemlerinde verinin istatistiksel modeli kullanılır; sıkıştırmanın kalitesi seçilen modele bağlıdır.
- Sözlük tabanlı sıkıştırma yöntemlerinde istatistiksel model kullanılmaz, işin içine olasılık girmez; değişken uzunluklu prefix kodlar da elde edilmez.

Sözlükteki her bir girdiye bazen sembol, bazen string denir: art arda gelmiş bir ya da birden fazla karakterdir; anlamlı ya da anlamsız olması önemli değildir. Bunlar token'lar üzerinden kodlanır.

Sözlükler statik ya da dinamik olabilir:

- **Statik sözlük:** sözlük sabittir; yeni bir şey eklemeye ya da çıkarmaya izin verilmez.
- **Dinamik sözlük:** sözlüğe yeni bir string eklenebilir; sözlük tamamen silinip baştan oluşturulabilir; az geçen semboller silinip yerine yeni gelenler konabilir.

12. Statik Sözlük Örneği ve Sıkıştırma Koşulu

Hoca, sözlük yöntemlerinin aslında böyle olmadığını, yalnız giriş amacıyla anlattığını belirterek basit bir statik sözlük örneği verdi. Slayttaki bilgiler (ekrandan, 01:40:00):

- Statik sözlüğün en basit örneği, İngilizce dokümanları sıkıştırmak için kullanılan İngilizce diline ait sözlüktür (kelimeler ek aldığı için Türkçe dili için uygun olmaz).
- Tanımları olmaksızın yarım milyon kelimeden oluşan bir sözlük düşünün.
- Bir kelime input stream'den okunur ve sözlükte aranır. Kelime sözlükte bulunuyorsa sözlükteki sırası ile output stream'e yazılır.
- Kelime sözlükte bulunmaz ise sıkıştırılmadan output stream'e aynen yazılır.
- Sözlükte yarım milyon kelime varsa, $2^{19} = 524.288$ kelime için index alanı olarak 19 bite ihtiyaç vardır.
- Output stream, kelimenin sözlükteki sıra değeri (index) ve ham kelimelerden oluşur. Bu iki değeri birbirinden ayırt etmek için ekstra bir bit kullanılır.

Hoca Türkçe örneğini şöyle verdi: "tanım" kelimesi sözlükte vardır, ama "tanımla", "tanımlar", "tanımları" yoktur. Bu yüzden sabit sözlükler Türkçe gibi sondan eklemeli dillerde kullanılamaz. İngilizcede ise ekler (ing, ed gibi) ayrı kodlanırsa kelimeler kök hâlleriyle sözlükte bulunur. Her kelimenin sözlükte bir sırası (index) vardır; örneğin "school" 121. satırdadır. 2^{18} yarım milyonun altında kaldığı için 19 bit gerekir; kelime 3 karakter de olsa 10 karakter de olsa 19 bitle kodlanır.

Kodlama düzeni (hocanın anlatımından):

- İlk bit **flag** bitidir: sözlükte varsa 0, yoksa 1.
- Flag 0 ise sonraki 19 bit okunur; bu, sözlükteki index'tir. Sözlükte olan bir kelime $1+19 = 20$ bitle kodlanır.
- Flag 1 ise ilk byte'ın kalan 7 biti kelimenin karakter uzunluğunu verir (örneğin 5). Ardından o kadar byte okunur. 5 karakterlik bir kelime için $8 + 5 \times 8 = 48$ bit gerekir.

Hoca, sözlükte bulunan kelimeler için yarıdan fazla sıkıştırma yapıldığını, ama önden, sondan ya da ortadan ek alan dillerde bu yöntemin mantıklı olmadığını, ancak sınırlı sayıda kelimeyle çalışılıyorsa kullanılabileceğini söyledi.

12.1 Kaç kelime sözlükte olmalı?

Slayttaki soru (ekrandan, 01:48:30): 5 karakter uzunluğunda N tane kelimeden kaç tanesi sözlükte bulunmalıdır ki sıkıştırmadan bahsedilebilsin?

Çözüm (derste verildi):

- Kodlama yapılmazsa her kelime $5 \times 8 = 40$ bittir; N kelime $40N$ bit tutar.

- Sözlükte olma (çakışma) olasılığı P olsun. Sözlükte olanlar 20 bit, olmayanlar $(1-P)$ 48 bitle kodlanır.
- Eşitlik: $N[20P + 48(1-P)] = 40N$.
- N'ler sadeleşir: $20P + 48 - 48P = 40 \Rightarrow -28P + 48 = 40 \Rightarrow 28P = 8$.
- $P = 8/28 \approx 0{,}29$ (python ile kontrol edildi: $0{,}2857$). Sıkıştırma için $P > 0{,}29$ olmalıdır.

Kelimelerin %29'u sözlükteyse sıkıştırılmış ve sıkıştırılmamış boyut eşittir; %30'un üstüne çıkılırsa sıkıştırma kazancı başlar. Slayta göre özel durumlarda birkaç yüz kelimeden oluşan statik sözlüklerin kullanılması daha iyi sonuç verir.

> **Ders sonrası not:** Slayttaki ara adım «9P = 8» hatalıdır; hoca derste bunu fark edip tahtada $28P = 8$ olarak düzeltti. Hoca 01:45:59'da kelime boyu için "40 byte" dedi; doğrusu $5 \times 8 = 40$ bittir.

13. String Sıkıştırma: Birden Fazla Sembolü Birlikte Kodlamak

Hoca, sözlük yöntemlerinde kullanılacak string'lerin ev, okul, çocuk gibi kelimeler değil, belirli karakterlerden oluşturulmuş semboller olduğunu hatırlattı. Soru şudur: iki karakteri birleştirmek mi, üç mü, dört mü daha iyi performans sağlar? Slayttaki ilke: sembollerden oluşan string'lere dayalı sıkıştırma yöntemleri, tek bir sembole dayalı yöntemlerden daha etkilidir (ekrandan, 01:50:00).

Tek sembol. İki sembollü bir alfabe: $a_1 \to P_1 = 0{,}8$, $a_2 \to P_2 = 0{,}2$.

- Ortalama olasılık değeri: $(0{,}8 + 0{,}2)/2 = 0{,}5$.
- Varyans (slayttaki tanımla, farkların mutlak değerlerinin toplamı): $|0{,}8-0{,}5|+|0{,}2-0{,}5| = 0{,}6$.
- Bu iki sembol 1 bitle kodlanır (birine 0, diğerine 1). Kodun ortalama uzunluğu 1 bit/sembol.

İkili string'ler. $2^2 = 4$ durum vardır: olasılıklar $0{,}64$; $0{,}16$; $0{,}16$; $0{,}04$. Huffman ile kodlanınca uzunluklar 1, 2, 3, 3 bit olur.

- Ortalama olasılık: $1/4 = 0{,}25$. Varyans: $0{,}78$.
- Ortalama kod uzunluğu: $1 \times 0{,}64 + 2 \times 0{,}16 + 3 \times 0{,}16 + 3 \times 0{,}04 = 1{,}56$ bit/string; string iki karakter olduğu için $0{,}78$ bit/sembol.

Üçlü string'ler (ekrandan, 01:53:40). $2^3 = 8$ kombinasyon vardır; her string'in olasılığı sembol olasılıklarının çarpımıdır:

String	Olasılık	Kod
$a_1a_1a_1$	$0{,}8 \times 0{,}8 \times 0{,}8 = 0{,}512$	0
$a_1a_1a_2$	$0{,}8 \times 0{,}8 \times 0{,}2 = 0{,}128$	100
$a_1a_2a_1$	$0{,}8 \times 0{,}2 \times 0{,}8 = 0{,}128$	101
$a_1a_2a_2$	$0{,}8 \times 0{,}2 \times 0{,}2 = 0{,}032$	11100
$a_2a_1a_1$	$0{,}2 \times 0{,}8 \times 0{,}8 = 0{,}128$	110
$a_2a_1a_2$	$0{,}2 \times 0{,}8 \times 0{,}2 = 0{,}032$	11101
$a_2a_2a_1$	$0{,}2 \times 0{,}2 \times 0{,}8 = 0{,}032$	11110
$a_2a_2a_2$	$0{,}2 \times 0{,}2 \times 0{,}2 = 0{,}008$	11111

- Ortalama olasılık: $1/8 = 0{,}125$.
- Varyans: $|0{,}512-0{,}125| + 3 \cdot |0{,}128-0{,}125| + 3 \cdot |0{,}032-0{,}125| + |0{,}008-0{,}125| = 0{,}792$.
- Ortalama Huffman kod uzunluğu: $1 \cdot 0{,}512 + 3 \cdot 3 \cdot 0{,}128 + 3 \cdot 5 \cdot 0{,}032 + 5 \cdot 0{,}008 = 2{,}184$ bit/string; $2{,}184/3 = 0{,}728$ bit/sembol.

String boyu	Olasılık varyansı	Ortalama kod uzunluğu (bit/sembol)
1	0,6	1
2	0,78	0,78
3	0,792	0,728

(Bütün değerler python ile doğrulandı.)

> **Ders sonrası not:** Hoca 01:53:22'de üçlü string'lerin ortalama olasılığına "0,25" dedi; slayttaki doğru değer $1/8 = 0{,}125$'tir. 01:52:05'te ikili string'lerin son terimi için "3 çarpı 0,4" dedi; doğrusu $3 \times 0{,}04$'tür.

String boyu arttıkça olasılıklar arasındaki varyans büyür, ama ortalama kod uzunluğu küçülür. Hoca, 6-7'li kombinasyonlarda bu değerin belki 0,6 gibi bir değere ineceğini, küçülmenin giderek azaldığını söyledi. Buradan sözlük yöntemlerinin neden performanslı olduğuna bağladı: tek karakterler yerine bir karakter topluluğu kodlanırsa, PPM'de olduğu gibi, daha iyi sıkıştırma elde edilir. PPM de, sözlük yöntemleri de daha büyük bir context'i kodladıkları için Huffman ve aritmetik kodlamaya göre daha performanslıdır.

Sözlüğün içinde birli, ikili, üçlü, dörtlü string'ler bulunur. Bunların anlamlı kelime olması ya da aynı uzunlukta olması gerekmez; sözlüğün oluşumuna bağlı olarak değişir.

14. LZ ve Türev Algoritmalar

Hoca sözlük yöntemlerinden yalnız LZ'yi (Lempel-Ziv) anlatacağını, sözlük kullanan diğer yöntemlerin hepsinin LZ77 ve LZ78'den türediğini söyledi. Slayttaki bilgiler (ekrandan, 01:57:10):

- Jacob Ziv ve Abraham Lempel isimli iki araştırmacı 1970'li yıllarda LZ77 ve LZ78 adını verdikleri ilk sözlük tabanlı sıkıştırma algoritmasını geliştirdi.
- Bu çalışma referans alınarak benzer yöntemler geliştirilmiştir:
  - LZSS → Storer-Szymanski
  - LZFG → Fiala-Greene
  - LZRW1 → Ross Williams
  - LZW → Welch
  - LZMW → Miller-Wegman
  - LZAP → All prefix
  - LZY → Yabba
  - LZP → Prediction

Hoca, LZ77'nin 1977'de ortaya çıktığını, 1978'de aynı kişilerin farklı bir algoritma ortaya attığını anlattı. Türevler, LZ'nin bazı parametreleri üzerinde oynar; tıpkı PPM'in A, B, C versiyonları gibi. Hoca LZ77 ve LZ78'i anlatıp en çok kullanılan LZ yöntemi olan LZW'den bahsedeceğini söyledi. Hocaya göre birçok arşiv sıkıştırma programında LZW kullanılır ve LZ77 ile LZ78'e göre çok daha performanslıdır.

15. LZ77: Kayan Pencere

LZ77'nin adı genelde **Sliding Window** (kayan pencere) yöntemi olarak geçer, çünkü algoritmada kayan bir pencere kullanılır. Input stream pencereye sokulur; pencereye giren her sembol kodlanır, pencerenin boyuna gelindiğinde kodlanmış olanlar dışarı çıkar. Türevler bu pencerenin boyutu, pencerede aramanın linear mı yoksa dairesel (circular queue) mi yapılacağı, kaynak kod alanının boyutu gibi konularda ayrılır.

Slayttaki bilgiler (ekrandan, 02:00:00):

- Bu yöntemdeki ana fikir, önceden input stream'den okunmuş olan bilgiyi sözlük gibi kullanmaktır.
- Kodlayıcı, input stream için bir pencere (window) oluşturur ve kodlanan input değerlerini sağdan sola doğru bu pencere içerisinde öteler.
- Bu yöntem kayan pencere (sliding window) mantığı üzerine kurulmuştur.
- Pencere iki parçadan oluşur. Sol taraftaki bölüm **Arama Alanı (Search Buffer)** olup mevcut sözlüktür; en son okunan ve kodlanan sembollerden oluşur.
- Pencerenin sağındaki bölüm ise **Kaynak Kod Alanı (Look-Ahead Buffer)** olup henüz kodlanmamış sembollerden oluşur.
- Pratikte arama alanı binlerce byte uzunluğunda iken kaynak kod alanı sadece onlarca byte uzunluğundadır.

Hoca, arama alanı ne kadar büyük olursa o kadar uzun eşleşmeler (match) yakalanacağını, ama uzunluğun bir sınırı olması gerektiğini ekledi.

15.1 En uzun çakışmanın bulunması ve token

Slayttaki örnek (ekrandan, 02:10:00): arama alanında «sir sid eastman easily t», kaynak kod alanında «eases sea sick seals» vardır.

- Kodlayıcı, kaynak kod alanının ilk sembolü olan «e»yi arama alanında sağdan sola doğru aramaya başlar.
- İlk «e» harfini «easily» kelimesinde yakalar. «e»nin offset değeri sağdan sayıldığında 8'dir.
- Algoritmanın amacı en uzun çakışmayı bulmaktır: e'nin ardından a var mı? Var (2 çakışma). s var mı? Var (3 çakışma). Dördüncü sembol: arama alanında i, kaynakta e; çakışma 3'te biter.
- Daha uzun bir çakışma olabilir mi diye arama sürer. Kodlayıcı «e» sembolünü ikinci kez «eastman» kelimesinde yakalar; burada da «eas» üç sembol olarak eşleşir (dördüncü sembol t ile e uyuşmaz).
- Bu işlem en uzun eşleşen string bulununcaya kadar, arama alanının sonuna kadar devam ettirilir.
- Kodlayıcı en uzun çakışmayı bulduktan sonra en son bulunan çakışmayı referans alarak **(16, 3, «e»)** token'ını hazırlar.

LZ77 prefix kod oluşturmaz, bir **token** oluşturur. Token'ın üç alanı vardır:

1. Çakışmanın olduğu adres (offset).
2. Çakışmanın uzunluğu (length).
3. Çakışan string'den sonra gelen karakter.

Örnekte offset 16, uzunluk 3 ve sonraki karakter e'dir. Hoca, aynı uzunlukta iki çakışma bulunduğunda hangisinin alınacağının algoritmanın "trick" yeri olduğunu söyledi. Bir türev ilk gördüğü çakışmayı alır; slayttaki yöntem en son bulunanı alır. Sadece en yakındaki çakışma alınırsa, geriye doğru aynı uzunlukta başka çakışmalar için fazladan kontroller yapılmaz.

> **Ders sonrası not:** Slayt offset'in "0'dan başlayarak" sayıldığını yazar; ama 8 ve 16 değerleri, arama alanının sağ ucundan 1'den başlayarak sayılan uzaklıktır («easily t» için t = 1, …, e = 8). Hoca da offset'in 0'dan ya da 1'den başlatılmasının türevden türeve değiştiğini söyledi.

Token output stream'e yazıldıktan sonra «e», «a», «s» ve token'daki dördüncü sembol «e» arama alanına girer. Arama alanı doluysa soldan dört karakter dışarı atılır. Pencerenin "kayan" olmasının sebebi budur: içine bir şeyler girdikçe diğer uçtan bir şeyler çıkar.

Offset alanının boyu sözlüğün (arama alanının) boyuna bağlıdır: offset için 1 byte ayrılırsa en fazla 255 yazılabilir ve arama alanına 256 karakter sığar. Normal bir metinde, kopyala-yapıştır yapılmadıkça çok uzun çakışmalar olmaz; bu yüzden length alanına genelde 1 byte verilir. Sonraki sembol de genelde 1 byte'tır. Türevlerde değişen genellikle offset alanının uzunluğu, sözlüğün boyu ve arama alanıyla kaynak kod alanı arasındaki orandır; bunlar deneysel çalışmalarla belirlenmiştir.

16. LZ77 Örnekleri ve Kod Çözme

16.1 Arama alanı boşken başlangıç

Algoritmanın başında arama alanı boştur, bu yüzden çakışma olmaz; offset ve length 0 olur. Slayttaki örnek (ekrandan, 02:14:50), girdi «sir sid eastman …»:

Adım	Arama alanı	Okunan	Token
1	(boş)	s	(0, 0, «s»)
2	s	i	(0, 0, «i»)
3	si	r	(0, 0, «r»)
4	sir	␣	(0, 0, «␣»)
5	sir␣	si + d	(4, 2, «d»)
6	sir␣sid	␣ + e	(4, 1, «e»)
7	sir␣sid␣e	a	(0, 0, «a»)

İlk dört karakter arama alanında olmadığı için (0, 0, sembol) olarak yazılır. Beşinci adımda s arama alanında vardır; «si» iki sembol çakışır, ardından d gelir. Slayttaki offset 4'tür: arama alanının sağ ucundan sayılınca ␣ = 1, r = 2, i = 3, s = 4. Hoca saymaya 0'dan başlayınca 3 bulduğunu, slaytın 1'den saydığını ve bunun türeve göre değiştiğini söyledi. Altıncı adımda boşluk 4 geride bulunur; boşluktan sonra e gelen bir çakışma yoktur, çakışma uzunluğu 1'dir ve sonraki sembol e'dir. a arama alanında yoktur: (0, 0, «a»). Hoca devamını sözlü olarak verdi: s gelir, arama alanında iki s vardır, ikisinin de ardından t gelmez; en son bulunan s'nin offset'i 10, uzunluk 1, sonraki sembol t'dir: (10, 1, «t»). Sonra m ile devam edilir.

Slayttaki gözlem: (0, 0, «s») ile yalnız tek bir sembol kodlanır ve bu iyi bir sıkıştırma sağlamaz.

16.2 Kod çözme

Hoca kod çözmeyi aynı token'lar üzerinden gösterdi. (0, 0, «s»): çakışma yok, s yazılır. Aynı şekilde i, r ve boşluk yazılır. (4, 2, «d»): 4 geriden başlayıp iki sembol alınır (si), sonuna d eklenir. (4, 1, «e»): 4 geriden bir sembol alınır (boşluk), sonuna e eklenir. Böylece «sir sid e» elde edilir.

Slayttaki bilgiler (ekrandan, 02:20:00):

- Kod çözücü kodlayıcıdan daha basittir.
- Kodlayıcının pencere boyutuna eşit bir buffer alanı oluşturulmalıdır.
- Kod çözücü token'ı okur, buffer'ında bulduğu çakışan sembolleri ve daha sonra da üçüncü alanı output stream'e yazar ve buffer alanını öteler.
- Bu yöntemler, bir kez sıkıştırılan çoğu kez açılarak kullanılan uygulamalar için elverişlidir (arşiv).
- Bu yöntemde okunan veri pattern'ları birbirine ne kadar yakınsa sıkıştırma performansı da o kadar iyidir.

Hoca kodlamanın neden zor olduğunu şöyle açıkladı: kodlayıcı her seferinde daha büyük bir çakışma var mı diye bakar ve en büyüğünü bulana kadar geri gider. Kod çözücü ise gördüğünü yazıp indeksi yürüterek çözer. Kod çözme hızlı olduğu için yöntem arşivlerde kullanılır.

17. LZ77'de Alan Uzunlukları ve Geliştirmeler

17.1 Token'ın bit uzunluğu

Slayttaki bilgiler (ekrandan, 02:14:50):

- Search buffer'ın uzunluğu S ise offset'in uzunluğu $\lceil \log_2 S \rceil$ olur.
- Pratikte search buffer birkaç bin byte uzunluğunda ise offset'in uzunluğu 10-12 bit olur.
- Look-ahead buffer'ın uzunluğu L ise length alanının uzunluğu $\lceil \log_2 (L-1) \rceil$ olur.
- Pratikte look-ahead buffer birkaç on byte'tan oluşuyorsa length alanının uzunluğu birkaç bit olur.
- Sembol alanının uzunluğu 8 bittir. Yaklaşık olarak bir token uzunluğu $11 + 5 + 8 = 24$ bittir.

Hoca, offset için 11 bitin arama alanında 2^{11} sembol tutulması anlamına geldiğini söyledi. Bir token 24 bit, yani 3 byte'tır. Dokümanın başında çakışma olmadığı için sıkıştırma yokmuş gibi görünür: örneğin yalnız s için (8 bitlik bir sembol) 24 bit harcanır. Arama alanı doldukça çakışmalar uzar ve belki 8-10, 15 karakter tek bir 24 bitlik token'la kodlanır. Bu yüzden ilk başlarda performans kötüdür.

17.2 LZ77'nin geliştirilmeleri

Slayttaki bilgiler (ekrandan, 02:20:00): LZ77 birçok araştırmacı tarafından geliştirilmiştir.

- Offset ve length alanları için değişken uzunluklar denenmiş.
- Her iki buffer'ın boyutları değiştirilmiş.
- Search buffer'ın boyu artırıldığında daha iyi çakışmalar elde edilmiş, ancak arama zamanı artmıştır.
- Kayan pencere mantığında **Linear Window** yerine **Circular Queue** mantığı kullanılmıştır.

Bunlara ek olarak türevler, çakışma bulunduğunda ilk bulunan mı yoksa en son bulunan çakışmanın mı kullanılacağına göre de ayrılır.

Linear ve circular arama. Linear pencerede yeni bir sembol geldiğinde pencere kaydırılır ve en soldaki sembol dışarı atılır. Circular yapıda kaydırma yapılmaz. Hoca bunu kendi örneğiyle anlattı: pencerede A, B, C, D olsun; start pointer A'yı, end pointer D'yi gösterir. K gelince sona eklenmez, A'nın yerine yazılır; artık start B'de, end K'dadır. Sonra J gelirse B'nin yerine yazılır; start C'ye, end J'ye geçer. Pencere dairesel döndürülür ve her yeni gelen, en eskinin yerine yazılarak onu dışarı çıkarır.

18. LZ78

LZ78, bir sene sonra aynı kişiler tarafından geliştirilmiştir ve algoritmayı oldukça değiştirir. Hocanın özetiyle:

- LZ77'de kayan pencere vardı ve ikiye ayrılıyordu: arama alanı ve kaynak kod alanı.
- LZ78'de kayan pencere, arama alanı ve kaynak kod alanı yoktur. Önceden karşılaşılmış string'lerden oluşan bir sözlük vardır. Sözlük boş başlar ve input stream'den gelen string'lerle doldurulur.
- Sözlüğün boyu kullanılabilir hafıza (memory) ile sınırlıdır.
- Kodlayıcı output stream'e **iki alanlı** bir token yazar: (string'in sözlükteki sırası, sembolün kendisi).

Algoritma (hocanın anlatımıyla): input stream'den bir w sembolü gelir ve sözlükte aranır. Sözlükte yoksa sözlüğün sonraki sırasına yerleşir ve token (0, w) yazılır. Varsa daha büyük bir çakışma bulmak için sonraki sembol k alınır, w ile birleştirilir ve wk sözlükte aranır. Varsa bir sonraki sembol eklenerek arama sürer. Bulunamazsa wk sözlüğe yeni sırayla yazılır; token'ın ilk alanına en son çakışan string'in sırası, ikinci alanına k yazılır.

18.1 Örnek

Girdi (ekrandan, 02:30:00): «sir sid eastman easily teases sea sick seals». Sözlüğün 0. sırası null'dur.

Sıra	String	Token	Açıklama
1	«s»	(0, «s»)	s sözlükte yok
2	«i»	(0, «i»)	i yok
3	«r»	(0, «r»)	r yok
4	«␣»	(0, «␣»)	boşluk yok
5	«si»	(1, «i»)	s var (1), si yok
6	«d»	(0, «d»)	d yok
7	«␣e»	(4, «e»)	boşluk var (4), ␣e yok
8	«a»	(0, «a»)	a yok
9	«st»	(1, «t»)	s var (1), st yok
10	«m»	(0, «m»)	m yok
11	«an»	(8, «n»)	a var (8), an yok
12	«␣ea»	(7, «a»)	␣e var (7), ␣ea yok
13	«sil»	(5, «l»)	si var (5), sil yok
14	«y»	(0, «y»)	y yok

(Tablo slayttaki gibidir; python ile doğrulandı.)

Hoca, dokümanın başında ilk alanların çoğunlukla 0 olduğunu, çünkü sözlüğün henüz yeni dolmaya başladığını söyledi. İlerledikçe uzun çakışmalar görülür. Bunu varsayımsal bir örnekle anlattı: girdinin sonuna «sir sit» gelsin ve sözlükte «sir␣» string'inin 25. sırada olduğunu düşünelim. s, si, sir, «sir␣» sözlükte bulunur; «sir␣s» bulunmaz. Token (25, «s») olur: 5 byte, 2 byte'lık bir token'la kodlanır. Index alanının boyu sözlüğün boyuyla orantılıdır: 1 byte ise sözlüğe 255 string yazılabilir, 2 byte ise çok daha fazla.

Kod çözme de çok kolaydır: (0, «s») ile s, (0, «i») ile i, sonra r ve boşluk yazılır. (1, «i»): 1. sırada s vardır; s yazılır, ardından i getirilir. (0, «d») ile d yazılır. (4, «e»): 4. sırada boşluk vardır; boşluk ve ardından e yazılır.

18.2 Sözlüğün trie ile tutulması ve dolması

LZ78'de de sözlükteki string'leri aramak için trie ağaçları kullanılır (ekrandan, 02:30:00 sonrası slayt). Root'tan alfabedeki bütün karakterler (boşluktan z'ye kadar) çıkar. Örneğin «sil» aranırken root'tan s bulunur, s'nin altında i, i'nin altında l bulunur; bu düğüm 13. sıradır. Kod çözerken 13 geldiğinde bunun «sil» olduğu bilinir ve arkasına token'daki sembol eklenir.

Sözlük hafızayla sınırlı olduğu için dolabilir. Dolunca yapılabilecekler:

- Sözlüğe hiç ekleme yapılmaz; sözlük donar (**frozen**). Yeni karakterler ASCII olarak gönderilir.
- Bütün sözlük silinir ve yeniden oluşturulur.
- En az kullanılan, kolay kolay tekrar etmeyen string silinir ve açılan yere yeni gelen eklenir.

19. LZW

19.1 Codebook

LZ'den türetilen algoritmalardan en önemlisi ve en iyi sonuç vereni LZW'dir. Geniş bir kodlama alanına sahiptir ve bir **codebook** kullanır. Hoca, buna sözlük de denebileceğini, ama kitaplarda bu yöntem anlatılırken "dictionary" yerine "codebook" geçtiği için kendisinin de codebook dediğini söyledi.

- Codebook dinamik olarak oluşturulur. Kodların boyu codebook'un uzunluğuna bağlıdır.
- Codebook'un boyu verilebilir: 8, 16 ya da 32 code word gibi. Örneğin 8 code word verildiyse 0'dan 7'ye kadar doldurulur ve sonra codebook'a yeni bir şey eklenemez. Sınır verilmemişse input stream bitene kadar büyür; ama yine de hafızayla sınırlıdır.
- Codebook'un başlangıcına alfabedeki semboller konur. Sonra input stream kullanılarak geliştirilir.

Örnek (ekrandan, 02:50:00): input stream «abbbaacbbabbaacacaabb», alfabe a, b, c. Başlangıçta 0 → a (0000), 1 → b (0001), 2 → c (0010); 3-13 sıraları boştur.

19.2 Codebook'un kurulması

Hoca önce yalnız codebook'un nasıl oluştuğunu, her adımı farklı renkle işaretleyerek gösterdi. Kural: okunan string codebook'ta varsa bir sonraki sembol eklenerek aranır; yoksa codebook'a yazılır ve son eklenen sembolden yeniden başlanır.

1. a var; ab yok → 3: ab. b'den başla.
2. b var; bb yok → 4: bb. b'den başla.
3. b var; bb var; bba yok → 5: bba. a'dan başla.
4. a var; aa yok → 6: aa. a'dan başla.
5. a var; ac yok → 7: ac. c'den başla.
6. c var; cb yok → 8: cb. b'den başla.
7. b, bb, bba var; bbab yok → 9: bbab. b'den başla.
8. b, bb, bba var; bbaa yok → 10: bbaa. a'dan başla.
9. a, ac var; aca yok → 11: aca. a'dan başla.
10. a, ac, aca var; acaa yok → 12: acaa. a'dan başla.
11. a, ab var; abb yok → 13: abb. b'den başla.
12. b var ve input bitti.

19.3 Output stream

Codebook kurulurken output stream'e de yazılır (ekrandan, 03:00:00). Her adımda codebook'ta bulunan en uzun string'in kodu gönderilir:

Okunan en uzun string	Gönderilen kod	Codebook’a eklenen
a	0	3: ab
b	1	4: bb
bb	4	5: bba
a	0	6: aa
a	0	7: ac
c	2	8: cb
bba	5	9: bbab
bba	5	10: bbaa
ac	7	11: aca
aca	11	12: acaa
ab	3	13: abb
b	1	(input bitti)

Output stream: **0 1 4 0 0 2 5 5 7 11 3 1** (python ile doğrulandı). Hoca bu tabloyu anlatırken birkaç yerde kodu karıştırdı (örneğin 02:58:20 civarında «bb» için kod vermeden geçti); slayttaki ve python ile bulunan sonuç yukarıdakidir.

Kodlar bit olarak gönderilir: 0 kodu 0000, 1 kodu 0001 gibi. Codebook’un boyu bilindiğinde her kod için kaç bit ayrılacağı bilinir; örneğin 32 elemanlı bir codebook’ta her kod 5 bitle yazılır. Sıkıştırırken bu bitler 8’lik gruplar hâlinde byte olarak alınır.

19.4 Kod çözme

Karşı taraf da sözlüğün 0’dan başladığını ve başında alfabenin (a, b, c) bulunduğunu bilir. 0 gelince a yazılır. Sonra 1 gelir: b. Demek ki a’nın arkasından b gelmiştir; a, ab codebook’ta olmadığı için yazılmıştı. ab codebook’a 3. sıraya eklenir. Sonra 4 gelir: 4, bb’dir; b’nin arkasından b gelmiş demektir ve bb 4. sıraya yerleşir. Kod çözücü de kodlayıcı gibi alfabeyle başlar, her okuduğu koddan sonra codebook’u aynı sırayla kurar; kodlayıcının yaptığının tersini yapar.

19.5 Codebook boyu sınırlıysa

Hoca, codebook 8 sembolle sınırlı olsaydı ne olacağını da anlattı. Codebook’a 7. sıradaki «ac»ye kadar ekleme yapılır; sonrakiler eklenemez. Bundan sonra sözlüğe ekleme yapılmaz; var olan en uzun string’in kodu yazılır ve kodlanamayan kısımdan yeniden başlanır. Açarken de ters işlem uygulanır.

> **Ders sonrası not:** Hoca 03:05:04’ten sonra bu durumu tahtada kodladı ve sonu “… 5, 7, 7, 3, 4” diye anlattı; kendisi de “karıştırdım birazcık, kaydırdım” dedi. Python ile hesaplanan çıkış **0 1 4 0 0 2 5 5 7 7 6 4**’tür: ilk altı kod aynıdır, sonra «bba» (5), «bba» (5), «ac» (7), «ac» (7), «aa» (6), «bb» (4) gelir. Kalan girdi «aabb»de «aa» 6 numarayla codebook’tadır, bu yüzden «ab» (3) değil 6 yazılır.

Örnek Sorular

Soru 1 (ekrandan, 00:30:00). «xyzzxyxyzzx» girdisini order-4 modeline göre PPM ile kodlamak için Order 4, 3, 2, 1, 0 context’lerini ve frekanslarını çıkarın. (b) Girdiye bir z daha gelirse tablo nasıl güncellenir?

Çözüm (derste verildi): (a) Order 0: x 4, y 3, z 4. Order 1: x→y 3; y→z 2, y→x 1; z→z 2, z→x 2. Order 2: xy→z 2, xy→x 1, yz→z 2, zz→x 2, zx→y 1, yx→y 1. Order 3: xyz→z 2, yzz→x 2, zzx→y 1, zxy→x 1, xyx→y 1, yxy→z 1. Order 4: xyzz→x 2, yzzx→y 1, zzxy→x 1, zxyx→y 1, xyxy→z 1, yxyz→z 1. (b) z 5 olur; x→z 1, zx→z 1, zzx→z 1, yzzx→z 1 eklenir (python ile doğrulandı).

Soru 2 (ekrandan, 00:50:00). «assanissimassa» girdisi için order-2 PPMC tablosunu escape sembolleriyle birlikte kurun.

Çözüm (derste verildi): bölüm 7’deki tablo. Escape frekansı, context’i izleyen farklı sembol sayısıdır; olasılıklar blok toplamına bölünerek bulunur (Order 0’da toplam 19).

Soru 3 (ekrandan, 01:00:00). Son context «sa» iken sırada n, s, m ya da d gelirse aritmetik kodlayıcıya hangi olasılıklar gönderilir ve kaç bit gerekir?

Çözüm (derste verildi): n: $1/2 \to 1$ bit. s: esc $1/2$, a→s $2/5 \to 2{,}32$ bit. m: esc $1/2$, esc $2/5$, m $1/19 \to 6{,}57$ bit. d: esc $1/2$, esc $2/5$, esc $5/19$, Order -1'de $1/28 \to 9{,}05$ bit (python ile doğrulandı).

Soru 4 (ekrandan, 01:48:30). Sözlükte olan kelime 20 bit, olmayan 5 karakterlik kelime 48 bitle kodlanıyor. 5 karakter uzunluğunda N tane kelimeden kaç tanesi sözlükte bulunmalıdır ki sıkıştırmadan bahsedilebilsin?

Çözüm (derste verildi): $N[20P + 48(1-P)] = 40N \Rightarrow 28P = 8 \Rightarrow P \approx 0{,}29$. Kelimelerin %29'undan fazlası sözlükte olmalıdır.

Soru 5 (ekrandan, 01:53:40). a_1 $(0{,}8)$ ve a_2 $(0{,}2)$ sembollerinden oluşan üçlü string'lerin olasılıkları ve Huffman kodları slayttaki gibi verilmiş olsun. Ortalama olasılık değerini, varyansı ve ortalama kod uzunluğunu bulun.

Çözüm (derste verildi): ortalama olasılık $1/8 = 0{,}125$; varyans $0{,}792$; ortalama Huffman kod uzunluğu $2{,}184$ bit/string, $2{,}184/3 = 0{,}728$ bit/sembol.

Soru 6 (ekrandan, 02:10:00). LZ77'de arama alanında «sir sid eastman easily t», kaynak kod alanında «eases sea sick seals» varken kodlayıcı hangi token'ı üretir?

Çözüm (derste verildi): «e» önce «easily»de (offset 8), sonra «eastman»da bulunur; iki yerde de «eas» 3 sembol çakışır. En son bulunan çakışma referans alınır: (16, 3, «e»).

Soru 7 (ekrandan, 02:14:50). Arama alanı boşken «sir sid eastman» girdisi LZ77 ile kodlanırsa ilk token'lar nelerdir?

Çözüm (derste verildi): (0,0,«s»), (0,0,«i»), (0,0,«r»), (0,0,«␣»), (4,2,«d»), (4,1,«e»), (0,0,«a»); hoca devamını sözlü olarak (10,1,«t») diye verdi.

Soru 8 (ekrandan, 02:30:00). «sir sid eastman easily teases sea sick seals» girdisi LZ78 ile kodlanırsa sözlüğün ilk 14 kaydı ve token'lar nelerdir?

Çözüm (derste verildi): bölüm 18.1'deki tablo: (0,s), (0,i), (0,r), (0,␣), (1,i), (0,d), (4,e), (0,a), (1,t), (0,m), (8,n), (7,a), (5,l), (0,y).

Soru 9 (ekrandan, 02:50:00 ve 03:00:00). Alfabesi a, b, c olan «abbbaacbbabbaacacaabb» input stream'i LZW ile kodlayın. Codebook'u ve output stream'i yazın.

Çözüm (derste verildi): codebook 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa, 11 aca, 12 acaa, 13 abb. Output stream: 0 1 4 0 0 2 5 5 7 11 3 1.

Soru 10 (derste sözlü, 03:05:04). Aynı girdi, codebook 8 kayıtla sınırlıyken nasıl kodlanır?

> **Çözüm (derste verilmedi):** Derste anlatılan sonuç karışık kaldığı için python ile hesaplandı. Codebook 0-7 (a, b, c, ab, bb, bba, aa, ac) dolduktan sonra ekleme yapılmaz. Output stream: 0 1 4 0 0 2 5 5 7 7 6 4.

Dersin Toparlaması

Dersin ilk bölümünde PPM tamamlandı. PPM, bir sembolün olasılığını kendisinden önce gelen N sembolün oluşturduğu context'e göre veren adaptif bir order-N modelidir. Order-N, N-1, …, 0 kolonlarından oluşan frekans tablosu sembol sembol kurulur; context bulunamazsa daha kısa context'e switch edilir ve bunun kod çözücüye bildirilmesi için her context bloğunun kendi escape sembolü aritmetik kodlayıcıya gönderilir. Hiç görülmemiş bir sembol Order -1'de $1/$alfabe olasılığıyla kodlanır; bu yüzden dokümanın başında sıkıştırma zayıftır. PPMA, PPMB ve PPMC yalnız escape kullanımında ayrılır, en yaygını PPMC'dir. Context'lere hızlı erişim için trie ve switch için pointer'lar kullanılır.

İkinci bölümde sözlük tabanlı yöntemlere geçildi. Bu yöntemlerde olasılık yoktur; string'ler bir sözlükte toplanır ve token'larla kodlanır. Birden fazla sembolü birlikte kodlamanın ortalama kod uzunluğunu düşürdüğü string sıkıştırma örneğiyle gösterildi. LZ77, arama alanı ve kaynak kod alanından oluşan kayan bir pencere kullanır ve (offset, length, sonraki sembol) token'ı üretir. LZ78 pencere yerine boş başlayan bir sözlük tutar ve (index, sembol) token'ı üretir.

LZW ise alfabeyle başlatılan bir codebook kullanır ve yalnız kodları gönderir; kod çözücü aynı codebook'u yeniden kurar. Hoca bir sonraki aşamada görüntü sıkıştırmasından ve JPEG'den bahsedeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.